Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана (национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ «Информатика и системы управления»

КАФЕДРА «Теоретическая информатика и компьютерные технологии»

Лабораторная работа № 14

по курсу «Методы оптимизации»

«Метод внешней точки для двумерного случая»

Студент группы ИУ9-82Б Шемякин В.А.

Преподаватель Посевин Д. П.

Москва 2026

1 Задание

Реализовать алгоритм внешней точки для думерного случая.

2 Результаты

Исходный код программы представлен в листинге 1.

```
import Pkg
using Plots, Printf, LinearAlgebra

gr()

f(x1, x2) = x1^2 + x2^2
g1(x1, x2) = 2 - x1
g2(x1, x2) = 2 - x2

function P_ext(x1, x2, r1, r2)
    f(x1, x2) + r1 * max(0.0, g1(x1, x2))^2 + r2 * max(0.0, g2(x1, x2))^2
end

function P_vis(x1, x2, r1, r2)
    f(x1, x2) + r1 * (2 - x1)^2 + r2 * (2 - x2)^2
end

function grad_P(x1, x2, r1, r2)
    d1 = 2x1
    d2 = 2x2
    if x1 < 2
        d1 += 2r1 * (x1 - 2)
    end
    if x2 < 2
        d2 += 2r2 * (x2 - 2)
    end
    [d1, d2]
end

function phi(t, x, d, r1, r2)
    xt = x .+ t .* d
    P_ext(xt[1], xt[2], r1, r2)
end

```

```
function line_search_ray(x, d, r1, r2; tmax=10.0, tol=1e-6)
    a = 0.0
    b = tmax
    q = (sqrt(5.0) - 1.0) / 2.0

    c = b - q * (b - a)
    d1 = a + q * (b - a)

    fc = phi(c, x, d, r1, r2)
    fd = phi(d1, x, d, r1, r2)

    while b - a > tol
        if fc < fd
            b = d1
            d1 = c
            fd = fc
            c = b - q * (b - a)
            fc = phi(c, x, d, r1, r2)
        else
            a = c
            c = d1
            fc = fd
            d1 = a + q * (b - a)
            fd = phi(d1, x, d, r1, r2)
        end
    end

    (a + b) / 2.0
end

function ray_minimize_ext(r1, r2; x0=[0.0, 0.0], tol=1e-8, maxiter
    =200)
    x = copy(x0)

    for _ in 1:maxiter
        g = grad_P(x[1], x[2], r1, r2)

        if norm(g) < tol
            return x
        end

        d = -g
        t = line_search_ray(x, d, r1, r2)
        x_new = x .+ t .* d

```

```
        if norm(x_new - x) < tol
            return x_new
        end

        x = x_new
    end

    x
end

function build_dynamic_penalties(R; n_phi=9)
    phis = range(pi/4, 0, length=n_phi)
    r1_list = Float64[]
    r2_list = Float64[]

    for     in phis
        r1 = R * (1 + cos(   )^2)
        r2 = R * (1 + sin(   )^2)
        push!(r1_list, r1)
        push!(r2_list, r2)
    end

    collect(phis), r1_list, r2_list
end

function get_history_ext(r1_list, r2_list; x0=[0.0, 0.0])
    history = []
    current_x = copy(x0)

    for (r1, r2) in zip(r1_list, r2_list)
        x_opt = ray_minimize_ext(r1, r2; x0=current_x)
        push!(history, (r1=r1, r2=r2, x1=x_opt[1], x2=x_opt[2]))
        current_x = x_opt
    end

    history
end

function plot_for_R_dynamic(x0, R, filename;
    x1_range=(0.0, 3.2),
    x2_range=(0.0, 3.2),
    n=180,
    zlim=(0, 120))

    phis, r1_list, r2_list = build_dynamic_penalties(R)
```

```
    history = get_history_ext(r1_list, r2_list; x0=x0)

    x1s = range(x1_range[1], x1_range[2], length=n)
    x2s = range(x2_range[1], x2_range[2], length=n)

    p = surface(
        x1s, x2s, zeros(n, n);
        alpha=0,
        xlabel=" x  ",
        ylabel=" x  ",
        zlabel="z",
        title="R = $R",
        camera=(28, 20),
        zlim=zlim,
        legend=:outertopright,
        colorbar=false,
        size=(1100, 800)
    )

    F = [f(x1, x2) for x2 in x2s, x1 in x1s]
    surface!(
        p, x1s, x2s, F;
        color=:lightgray,
        alpha=0.22,
        label="f( x  , x  )",
        colorbar=false
    )

    idxs = [1, Int(cld(length(r1_list), 2)), length(r1_list)]
    surf_colors = [:deepskyblue, :gold, :orangered]

    for (k, idx) in enumerate(idxs)
        r1 = r1_list[idx]
        r2 = r2_list[idx]
           = phis[idx]
        Z = [P_vis(x1, x2, r1, r2) for x2 in x2s, x1 in x1s]

        label_str = @sprintf("   = %.2f",    )

        surface!(
            p, x1s, x2s, Z;
            color=surf_colors[k],
            alpha=0.34,
            label=label_str,
            colorbar=false
```

```
        )
    end

    x1_traj = [h.x1 for h in history]
    x2_traj = [h.x2 for h in history]
    z_traj = [P_vis(h.x1, h.x2, h.r1, h.r2) for h in history]

    z0 = P_vis(x0[1], x0[2], r1_list[1], r2_list[1])

    plot!(
        p,
        [x0[1], x1_traj[1]],
        [x0[2], x2_traj[1]],
        [z0, z_traj[1]];
        lw=2.5,
        ls=:dash,
        color=:navy,
        label=""
    )

    plot!(
        p,
        x1_traj, x2_traj, z_traj;
        lw=4,
        color=:magenta,
        marker=:circle,
        markersize=5,
        label="                    "
    )

    scatter!(
        p,
        [x0[1]], [x0[2]], [z0];
        ms=16,
        color=:black,
        marker=:star5,
        label="          "
    )

    scatter!(
        p,
        [2.0], [2.0], [8.0];
        ms=13,
        color=:green,
        marker=:diamond,
```

```
        label="(2,2,8)"
    )

    savefig(p, filename)

    x_last = history[end].x1
    y_last = history[end].x2
    println(@sprintf("R = %.1f -> x*      (%.6f, %.6f), f(x*) = %.6f", R, x_last, y_last, f(x_last, y_last)))
end

mkpath("images")

x0 = [0.0, 0.0]
R_list = [10.0, 20.0, 50.0, 100.0, 500.0, 1000.0]

for R in R_list
    filename = @sprintf("images/graphic_R_%0.0f.png", R)
    plot_for_R_dynamic(x0, R, filename)
end
```

Результат представлен на рисунках 1– 2.

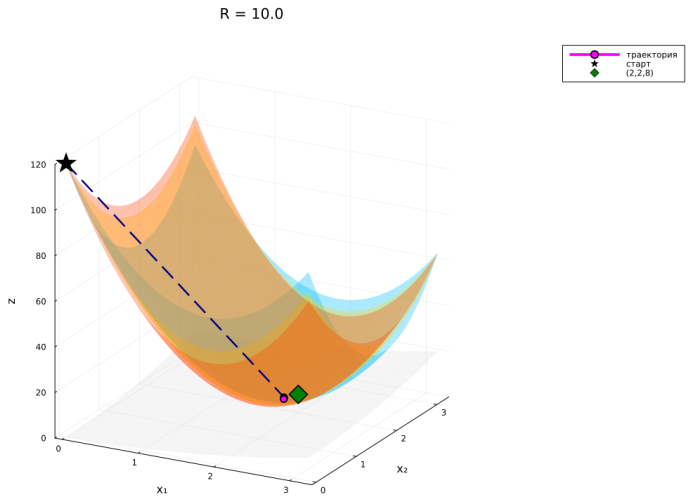

Рис. 1 — Результат

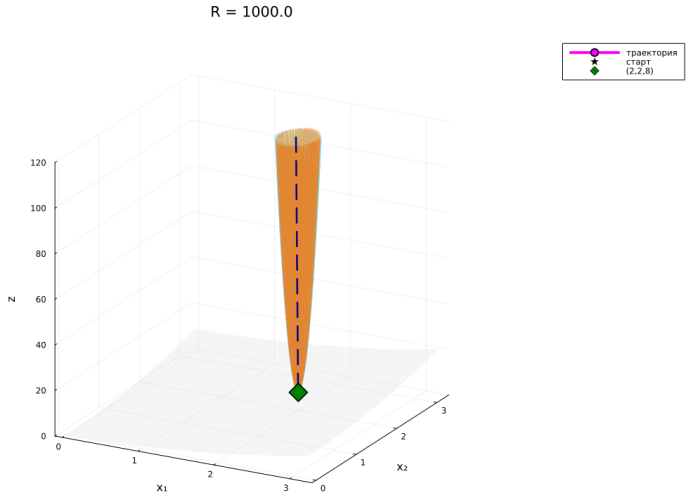

Рис. 2 — Результат